Praktikum 5

Viktoriia Bazhinova

Adib Gharibe

Der Docker-Container kann mit dem Befehl “docker build -t bs-praktikum5 .” erstellt werden.

```
┌──(kali㉿kali)-[~/mydocker]
└─$ sudo docker build -t bs-praktikum5 .

[sudo] password for kali:
[+] Building 59.5s (10/10) FINISHED
 => [internal] load build definition from Dockerfile
 => => transferring dockerfile: 399B
 => [internal] load metadata for docker.io/library/debian:bookworm
 => [internal] load .dockerignore
 => => transferring context: 2B
 => [1/5] FROM docker.io/library/debian:bookworm@sha256:d42b86d7e24d78a33edcf1ef4f65a20e34acb1e1abd53cabc3f7cdf769fc4082
 => => resolve docker.io/library/debian:bookworm@sha256:d42b86d7e24d78a33edcf1ef4f65a20e34acb1e1abd53cabc3f7cdf769fc4082
 => => sha256:d42b86d7e24d78a33edcf1ef4f65a20e34acb1e1abd53cabc3f7cdf769fc4082 8.52kB / 8.52kB
 => => sha256:ef6ef067c154c1d1f43a5596516b08ccfc961fc8809c3c1e7002615ce26b1a28 1.02kB / 1.02kB
 => => sha256:b3a422523a1134fdfb331bcc41df232c4fb2bf3f0444e3c2b7e675db095a6626 453B / 453B
 => => sha256:c1995213564325caf7e52ecd95fe4435c70b03eb94c674ac15706733986b86e0 48.49MB / 48.49MB
 => => extracting sha256:c1995213564325caf7e52ecd95fe4435c70b03eb94c674ac15706733986b86e0
 => [internal] load build context
 => => transferring context: 1.20kB
 => [2/5] RUN apt update && apt install -y build-essential
 => [3/5] WORKDIR /app
 => [4/5] COPY main.cpp .
 => [5/5] RUN g++ -O2 -o myprogram main.cpp
 => exporting to image
 => => exporting layers
 => => writing image sha256:76f558fc5c3a91ae8c033afba7bdab2b66dd8459dea5b1042872ea89e7fb3946
 => => naming to docker.io/library/bs-praktikum5
```

Ohne-Limitierung (Szenario 1a):

```
CONTAINER ID   NAME         CPU %     MEM USAGE / LIMIT     MEM %     NET I/O       BLOCK I/O       PIDS
1af67400a151   szenario1a   113.60%   1.533GiB / 1.921GiB   79.83%    876B / 126B   317MB / 996MB   3
```

```
File  Actions  Edit  View  Help
CONTAINER ID   NAME         CPU %     MEM USAGE / LIMIT     MEM %     NET I/O       BLOCK I/O       PIDS
ae4a03de1439   szenario1a   38.12%    1.581GiB / 1.921GiB   82.31%    946B / 126B   337MB / 1.28GB   3
```

Das Verhalten zeigt:

- Anfangs ist die **CPU-Last sehr hoch** -> aktive Berechnungen.
- Später sinkt die CPU-Last ->Programm befindet sich wahrscheinlich im Abschluss.
- **Speicherverbrauch steigt kontinuierlich** und bleibt stabil, da der vector nicht gelöscht wird.

Das Programm erzeugt **nachweislich hohe CPU-Last ca 113%** und **Speicherverbrauch von über 1,5 GiB**.

Der Container wurde **ohne Ressourcenlimits gestartet** und zeigt **unbegrenzten Zugriff auf Rechenleistung**, was der Aufgabenstellung "unmodifizierter Container" entspricht.

CPU-Limitierung (Szenario 2a):

Zur Limitierung des CPU-Anteils wurde anschließend der Container mit dem folgenden Parameter gestartet:

```
┌──(kali㉿kali)-[~]
└─$ sudo docker run -d --name szenario2a --cpus="0.5" bs-praktikum5
```

Dadurch wurde der Container **auf 50 % eines Kerns** begrenzt.

Beim Start des Containers szenario2a mit --cpus="0.5" wurde im docker inspect der Wert NanoCpus korrekt auf 500000000 gesetzt.
Das entspricht **genau einem CPU-Limit von 0.5 CPU**.

```
┌──(kali㉿kali)-[~]
└─$ sudo docker inspect szenario2a | grep -i cpu

            "CpuShares": 0,
            "NanoCpus": 500000000,
            "CpuPeriod": 0,
            "CpuQuota": 0,
            "CpuRealtimePeriod": 0,
            "CpuRealtimeRuntime": 0,
            "CpusetCpus": "",
            "CpusetMems": "",
            "CpuCount": 0,
            "CpuPercent": 0,
```

```
File  Actions  Edit  View  Help
CONTAINER ID   NAME        CPU %    MEM USAGE / LIMIT     MEM %    NET I/O     BLOCK I/O     PIDS
77ddf526178c   szenario2a  59.07%   1.369GiB / 1.921GiB   71.25%   806B / 126B   4.85MB / 0B   3
```

Beim Start mit --cpus="0.5" war die CPU-Auslastung auf ca. 50–60 % begrenzt, während sie ohne Limit bei über 100 % lag.
Dies bestätigt, dass die CPU-Limitierung durch den Parameter erfolgreich greift.

2a Szenario mit --cpu-shares 256

Die Gewichtung von Containern über den Parameter --cpu-shares bestimmt die Verteilung der CPU-Ressourcen zwischen konkurrierenden Containern.
Im durchgeführten Test wurden zwei Container mit CPU-Bindung auf denselben Kern (--cpuset-cpus=0) gestartet.
Container A erhielt die Standardgewichtung (1024), Container B nur 512.

```
┌──(kali㉿kali)-[~]
└─$ sudo docker run -dit --cpuset-cpus 0 --cpu-shares 1024 --name cont1 bs-praktikum5
1b231f04c8730af905896b5bb34d4549c5fda75c02673e6309d613bf2d3a1300

┌──(kali㉿kali)-[~]
└─$ sudo docker run -dit --cpuset-cpus 0 --cpu-shares 512 --name cont2 bs-praktikum5
3d739229f062b2f2bc948fe9552487ecdeba79e14115f44839f1b49c9764df57
```

Beobachtungen aus docker stats:

- Container mit höherem Gewicht (1024) erhält ca. 62 % CPU-Zeit
- Container mit geringerem Gewicht (512) erhält ca. 32 % CPU-Zeit

```
CONTAINER ID   NAME      CPU %     MEM USAGE / LIMIT     MEM %     NET I/O         BLOCK I/O       PIDS
9f338940ad9a   cont1     61.65%    919.9MiB / 1.921GiB   46.76%    1.31kB / 126B   332MB / 643MB   3
0cac5a9646ac   cont2     31.94%    522MiB / 1.921GiB     26.54%    1.16kB / 126B   122MB / 207MB   3
```

Das Verhalten bestätigt die relative Gewichtung bei CPU-Shares.

Bei gemeinsamer Nutzung desselben CPU-Kerns und unterschiedlichen --cpu-shares ergibt sich ein Verhältnis der CPU-Auslastung entsprechend dem Verhältnis der Gewichte (2:1).

Es handelt sich nicht um absolute Limits, sondern um ein Verteilungssystem, das nur bei aktiver Konkurrenz greift.

Ohne gemeinsame CPU-Zuweisung würde diese Gewichtung keinen messbaren Effekt haben.

Hauptspeicher-Limitierung (Szenario 2b):

In Szenario 2b wurde der Container mit dem Parameter --memory="512m" gestartet.

```
┌──(kali㉿kali)-[~]
└─$ sudo docker run -d --name szenario2b --memory="512m" bs-praktikum5
```

```
┌──(kali㉿kali)-[~]
└─$ sudo docker inspect szenario2b | grep -i memory

            "Memory": 536870912,
            "MemoryReservation": 0,
            "MemorySwap": 1073741824,
            "MemorySwappiness": null,
```

Durch docker inspect wurde bestätigt, dass der Container szenario2b erfolgreich auf **512 MB Hauptspeicher** begrenzt wurde:
Der Eintrag "Memory": 536870912 entspricht exakt 512 MB RAM.

In docker inspect war zusätzlich "MemorySwap": 1073741824 gesetzt, was bedeutet: **Docker erlaubt weitere 512 MB Swap**.

Swapping bezeichnet das Auslagern von nicht mehr aktiv genutzten Speicherbereichen auf die Festplatte, wenn der RAM-Limit erreicht ist.
Dies erlaubt dem Container, **weiter zu arbeiten**, obwohl der physische Speicher bereits ausgelastet ist.

```
File  Actions  Edit  View  Help
CONTAINER ID   NAME         CPU %     MEM USAGE / LIMIT   MEM %     NET I/O       BLOCK I/O       PIDS
6584f1b6bb83   szenario2b   73.85%    511.8MiB / 512MiB   99.97%    876B / 126B   252MB / 728MB   3
```

In docker stats wurde ein **Speicherverbrauch von 511.8 MiB bei 99.9 %** angezeigt. Das Programm lief weiter - **Swapping war aktiv möglich**.

Hätte Docker keinen Swap erlaubt (--memory-swap=512m), wäre der Container bei Erreichen des RAM-Limits **abgestürzt** (OOM: Out of Memory).
In unserem Fall konnte der Container durch Swapping noch weniger Sekunden weiterarbeiten.

Szenarioanalyse: Ressourcenlimitierung

Zur Analyse der Ressourcenverteilung und -begrenzung wurden drei Container mit unterschiedlichen Parametern gestartet:

```
┌──(kali㉿kali)-[~]
└─$ sudo docker run -d --name cont1 --cpu-shares=1024 --cpuset-cpus=0 bs-praktikum5
sudo docker run -d --name cont2 --cpu-shares=512 --cpuset-cpus=0 bs-praktikum5
sudo docker run -d --name cont3 --memory="512m"  bs-praktikum5
```

Ziel

- Vergleich der CPU-Leistung bei zwei Containern mit unterschiedlichem CPU-Gewicht (Shares).
- Beobachtung der Speicherauslastung eines Containers mit festem Speicherkontingent.

CONTAINER ID	NAME	CPU %	MEM USAGE / LIMIT	MEM %	NET I/O	BLOCK I/O	PIDS
4e87c211110d	cont1	40.40%	548.6MiB / 1.921GiB	27.89%	1.36kB / 126B	73MB / 243MB	3
471523dc1e85	cont2	22.28%	441.3MiB / 1.921GiB	22.43%	1.02kB / 126B	45.9MB / 154MB	3
32fc3ac9499e	cont3	81.97%	449.8MiB / 512MiB	87.85%	932B / 126B	132MB / 519MB	3

- cont1 nutzt mehr CPU als cont2, was dem Verhältnis der CPU-Shares (1024:512 = 2:1) entspricht.
- cont3 nutzt deutlich mehr CPU, da kein CPU-Limit gesetzt wurde.
- cont3 stößt fast an seine Speichergrenze, was korrekt dem gesetzten Limit entspricht. cont3 erreichte schnell 450 MiB (ca. 88 % Auslastung) und zeigte in späterer Beobachtung 0 % CPU- und Speicherauslastung.
  Dies weist darauf hin, dass der Containerprozess gestoppt wurde.
- Die anderen Container sind nicht begrenzt und nutzen dynamisch Speicher, aber weit unterhalb des Limits.